

Pytőfi Szótár – Petőfi-versek szótárkezelő alkalmazás

Készítette: Gál Vivien Viktória – FKT6WE

1. A program célja

A Pytőfi Szótár egy Python nyelven írt program, amely Petőfi Sándor verseinek feldolgozásával szógyakorisági szótárt készít. A program képes:

- Versek beolvasására és tárolására (cím, szerző, dátum, szöveg)
 - A szöveg feldolgozására: írásjelek eltávolítása, szavak kisbetűsítése
 - Szavak keresésére, megtekintésére és gyakoriság szerinti rendezésére
 - Rendező algoritmusok (QuickSort, Kupacrendezés, Buborékrendezés, Shell Sort) demonstrálására
-

2. Környezet és futtatás

A program konzolos alkalmazás, Python nyelven készült.

Futtatás:

A projekt fájlokat tartalmazó mappában meg kell jelenjenek az alábbi elemek

- main.py
- pytofiapp.py
- menu.py
- poemmanager.py
- dictionarymanager.py
- sortingalgorithms.py
- stopwords.txt (stop-szavak listája)
- /poems/ mappa (a versfájlokkal)

A program indítása parancssorból: python main.py

Elindulás után megjelenik a főmenü, ahol a felhasználó a billentyűzetről (1–7) választhat menüpontot.

3. Adatszerkezetek összefoglalása

A program a feldolgozott adatok tárolására JSON fájlt használ, amely összetett, egymásba ágyazott szerkezetet tartalmaz

data.json struktúrája

A data.json fájl a belső szótár fő tárolója. Fő kulcsai:

titles: lista

- A versek címeit tartalmazza.
- Az index (0, 1, 2, ...) a vers azonosítója (poem_id).

poems: szótár (dict)

- Kulcs: a vers azonosítója sztringként (pl. "0", "1", "2").
- Érték: egy belső szótár a vers metaadataival:

```
{
    "title": "vers címe",
    "author": "Petőfi Sándor",
    "date": "megírás dátuma vagy évszáma",
    "source": "forrás megjelölése"
}
```

words: szótár (dict)

- Kulcs: egy szó (kisbetűs sztring).
- Érték: belső szótár a szó statisztikáival:

```
{
    "count": előfordulások száma (int),
    "poems": [poem_id_1, poem_id_2, ...]
}
```

szotar export.json struktúrája

A DEF_export_dictionary() egy külön export fájlt hoz létre szotar_export.json néven. Ennek szerkezete hasonló:

- titles – versek címei
- poems – versek metaadatai
- words – rendezett szótár (a kiválasztott algoritmussal rendezve)

Az exportált fájl célja, hogy a rendezett szótár külön fájlként, könnyen böngészhető formában megmaradjon.

stopwords.txt struktúrája

- Egysoros, kisbetűs szavakat tartalmazó szövegfájl.
- Minden sor egy „stop szó”, amelyet a program nem vesz figyelembe a szótárépítés során (pl. „és”, „hogy”, „az”, ...).

A dictionarymanager.py modul DEF_load_stopwords() függvénye tölti be.

/poems/ mappa

A versek szövegfájljai ebben a mappában találhatók, .txt kiterjesztéssel.

Egy fájl felépítése:

1. sor: Cím
2. sor: Szerző
3. sor: Dátum
4. sortól: a vers törzsszövege

A CL_PoemManager ezen szerkezetet feltételezi a beolvasáskor, a metaadatokat kiválasztja, a törzsszöveget továbbadja a szótárkezelőnek.

Felhasználói dokumentáció

A program indítása

- A programot parancssorból lehet indítani: `python main.py`
- Indítás után megjelenik a főmenü, ahol a felhasználó szám beírásával (1–7) választhat.

Főmenü menüpontjai – felhasználói szemmel

1 – Vers hozzáadása

- A program kilistázza a `/poems/` mappában található verseket.
 - A felhasználó sorszám alapján kiválasztja, melyik verset szeretné hozzáadni a szótárhoz.
 - A program beolvassa a vers címét, szerzőjét, dátumát és szövegét.
 - Csak „Petőfi Sándor” szerzőjű verseket fogad el, duplikált cím esetén hibát jelez.
 - A vers szavai bekerülnek a szótárba, a gyakoriságuk frissül, és a módosítás mentésre kerül a `data.json` fájlba.
-

2 – Rendezési algoritmusok összehasonlítása

- A program a `data.json` fájlból betölti az aktuális szótárat.
 - Mind a négy rendezési algoritmust (QuickSort, HeapSort, BubbleSort, ShellSort) lefuttatja ugyanazon adatokon.
 - A futási időket egymás mellett, vizuálisan (csillagokkal jelölve) jeleníti meg, hogy látható legyen, melyik algoritmus mennyire gyors.
-

3 – Szótár mentése fájlba

- A felhasználó kiválaszthatja, hogy melyik rendezési algoritmussal szeretné rendezni a szótárat.
 - A program a kiválasztott algoritmust használva rendezi a words struktúrát.
 - Az eredményt `szotar_export.json` néven menti el.
 - A mentés sikerességéről visszajelzést ad.
-

4 – Szótár megtekintése

- A program beolvassa a `szotar_export.json` fájlt.
- A felhasználó megadhat egy kezdőbetűt (vagy ENTER-rel kihagyhatja).
- A program listázza a megfelelő szavakat, a hozzájuk tartozó előfordulásszámmal és azzal, hogy mely versekben szerepelnek.
- Kimenet: táblázatos formában: „Szó – Db – Művek”.

5 – Szó keresése

- A felhasználó egy keresendő szót vagy szórészletet adhat meg.
 - A program többféle keresési módot támogat:
 - pontos szavas keresés,
 - szórészlet alapján,
 - előtag (prefix) alapján.
 - A találatoknál megjelenik:
 - hányszor fordul elő az adott kifejezés,
 - mely versekben található meg.
-

6 – Versek megtekintése

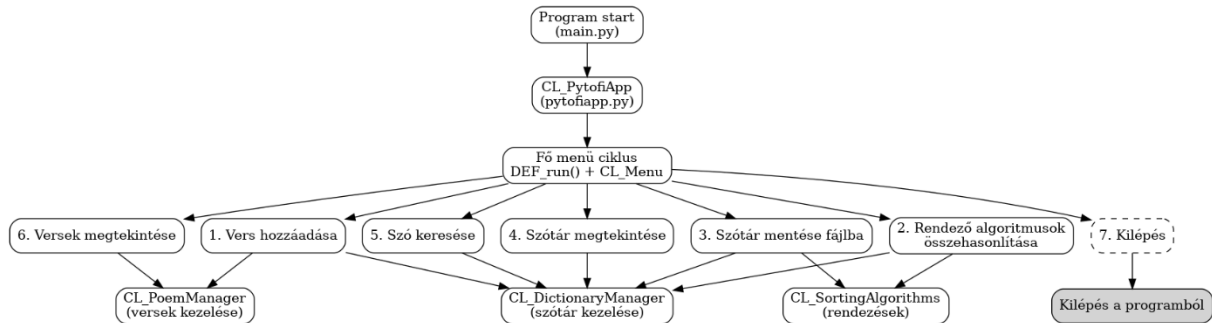
- A program kilistázza a /poems/ mappában található verseket.
 - A felhasználó sorszám alapján kiválaszt egy verset.
 - A program kiírja a konzolra a vers teljes tartalmát (cím, szerző, dátum, törzsszöveg).
-

7 – Kilépés

- A program befejezi a futását, visszatér a parancssorba.

Programozói dokumentáció

Folyamatábrája:



menu.py (Menü)

A menu.py modul tartalmazza a program felhasználói menürendszerét.

Feladata, hogy a felhasználó elé rendezett, átlátható menüpontok kerüljenek, és a program minden funkciója egyszerűen elérhető legyen.

CL_Menu osztály tartalma:

1. DEF_show()

- Színes, unicode keretes menüt jelenít meg a képernyőn.
- A menü elemei:
 1. Vers hozzáadása
 2. Rendezési algoritmusok összehasonlítása
 3. Szótár mentése
 4. Szótár megtekintése
 5. Szó keresése
 6. Versek megtekintése
 7. Kilépés
- A pyconio könyvtárat használja a színezéshez és a képernyő törléséhez.

2. DEF_get_choice()

- Bekéri a felhasználó menüpontválasztását.
- Hibakezelést tartalmaz:
 - ha nem számot ad meg,
 - ha 1–7-en kívül esőt ad meg.
- Csak érvényes menüpont esetén tér vissza számmal.

3. DEF_wait_for_enter()

- Leállítja a programot, és megvárja, amíg a felhasználó megnyomja az Entert.
- Minden alfunkció végén használatos, hogy a képernyő ne tűnjön el azonnal.

poemmanager.py

A poemmanager.py modul irányítja a versek fájlrendszerből való betöltését.

A Pytőfi alkalmazás minden verset a /poems/ könyvtárból olvas be.

CL_PoemManager osztály tartalma:

1. Inicializálás

- A konstruktor létrehozza a poems mappát, ha nem létezik.

2. DEF_list_poems()

- Kilstázza a mappában lévő .txt kiterjesztésű fájlokat. (A verseket)
- Minden fájl egy lehetséges vers, amely betölthető.

3. DEF_choose_poem()

- Sorszám alapján kiválasztható a betölteni kívánt vers.
- Hibakezelés kétféle módon:
 - ha a felhasználó nem számot ad meg,
 - ha a szám nem szerepel a listában.

4. DEF_read_poem(filename)

- Beolvassa a választott fájl teljes tartalmát UTF-8 kódolással.
- A fejléc három sorát (Cím/Szerző/Dátum) a fő alkalmazás dolgozza fel később.

dictionarymanager.py

Ez a modul a program gerince, kezeli a szótárat, feldolgozza a betöltött szövegeket, menti a JSON fájlba és támogatja a keresést és a rendezést.

CL_DictionaryManager osztály tartalma:

1. Inicializálás

- Betölti a JSON adatfájlt, ha létezik.
- Inicializálja a következő kulcsokat:
 - titles – versek címei,
 - poems – versek metaadatai,
 - words – a szótár tartalma.

- Betölti a stopwords listát (stopwords.txt).

2. DEF_add_text(text, title, author, date, source)

Ez a szótárépítés központi része.

Főbb műveletek:

1. Duplikáció ellenőrzése

- Ugyanaz a cím nem adható hozzá kétszer.

2. Szerző ellenőrzése

- Csak „Petőfi Sándor” betölthető.

3. Szótisztítás és feldolgozás

- Írásjelek eltávolítása,
- Ékezetek meghagyása,
- Kisbetűsítés,
- Stopwords szűrése.

4. Szótárfrissítés

- Számláló növelése,
- Vers azonosítójának rögzítése.

3. DEF_save()

- A teljes szótárat JSON formátumban menti a data.json fájlba.

4. DEF_search(word)

- Visszaadja egy szó statisztikáit, ha szerepel a szótárban.

5. DEF_sort_words()

- Bekéri, melyik algoritmussal rendezzen.
- Elérhető algoritmusok:
 - QuickSort
 - HeapSort
 - BubbleSort
 - ShellSort
- Mérés eredményként megjeleníti a futási időt.

6. DEF_compare_algorithms()

- Mind a négy rendezést lefuttatja ugyanazon adaton.

- Listázza és összeveti a futási időket.
-

sortingalgorithms.py

A rendezésekért felelős modul, amely a magyar ABC sajátosságaihoz igazított karakternormalizációval dolgozik.

CL_SortingAlgorithms osztály tartalma:

1. normalize_hu(word)

- Lépcsőzetesen súlyozza a magyar ékezetes betűket:
 - a0 < á1
 - e0 < é1
 - o0 < ó1 < ö2 < ő3
- Enélkül a Unicode helytelenül rendezné a magyar szavakat.

2. Rendezési algoritmusok

QuickSort

A QuickSort egy hatékony, rekurzív rendezési algoritmus, amely az „oszd meg és uralkodj” elvet alkalmazza. Működése során először kiválaszt egy elemet a rendezendő listából, amelyet pivotnak nevezünk. A pivot szerepe az, hogy a többi elemet hozzá viszonyítva két csoportra bontsa: azokra, amelyek kisebbek nála, és azokra, amelyek nagyobbak. Ezután a QuickSort külön-külön, önmagába visszatérve rendezi a pivotnál kisebb és nagyobb elemekből álló részeket. A folyamat mindaddig ismétlődik, amíg a listák már csak egy elemet tartalmaznak, vagy üresek, ekkor ugyanis már rendezettnek tekinthetők. Végül a rendezett „kisebb”, „pivot” és „nagyobb” részeket összefűzi, így áll elő a teljesen rendezett lista. A módszer gyorsaságát az adja, hogy a lista méretét minden lépésben két kisebb részre bontja, ezért átlagosan nagyon hatékonyan működik.

HeapSort

A HeapSort egy hatékony rendezési algoritmus, amely a kupacnak nevezett speciális adatszerkezetet használja fel a rendezéshez. A működése úgy kezdődik, hogy a bemeneti listából egy úgynevezett max-heapet épít, amely olyan bináris fa, ahol minden szülő mindig nagyobb vagy egyenlő a gyerekeinél. Ez azt jelenti, hogy a legnagyobb elem mindig a kupac tetején, vagyis a lista legelső pozíciójában található. Miután a max-heap elkészült, az algoritmus kicseréli a legnagyobb elemet a lista utolsó elemével, majd azt „kivonja” a kupacból, mert már a helyére került a rendezett sor végére. Ezután a kupac mérete eggyel csökken, és a maradék elemekből újra helyreállítja a max-heap szerkezetet, hogy ismét a legnagyobb elem kerüljön a tetejére. Ezt a folyamatot addig ismétli, amíg az összes elem a megfelelő helyére nem kerül. A HeapSort nagy előnye, hogy mindig $O(n \log n)$ időben működik, és nem igényel további nagy memóriaterületet, mivel a rendezést többnyire a bemeneti listán belül hajtja végre.

BubbleSort

A BubbleSort egy egyszerű, de viszonylag lassú rendezési algoritmus, amely úgy működik, hogy a listát többször végigjárja, és egymás mellett álló elemeket hasonlít össze. Ha egy pár rossz sorrendben van – vagyis az előbb álló elem nagyobb a következőnél –, akkor a két elemet felcseréli. A folyamat közben a legnagyobb elemek fokozatosan „felúsznak” a lista végére, innen ered a módszer neve is (bubble = buborék). Miután egy teljes bejárás végére a legnagyobb elem a helyére kerül, az algoritmus egyre rövidebb részeket vizsgál, hiszen a végére kerülő elemeket már nem kell tovább mozgatni. A BubbleSort ezt a páronkénti összehasonlítást és cserét addig ismétli, amíg az egész lista rendezetté nem válik. Bár könnyen érthető és egyszerűen megvalósítható, a működése miatt általában lassú, mivel átlagos és legrosszabb esetben is $O(n^2)$ időt igényel, emiatt nagy adathalmazokon nem hatékony.

ShellSort

A ShellSort a beszűrő rendezés továbbfejlesztett, hatékonyabb változata, amely úgy gyorsítja a rendezést, hogy az elemeket nem csak a közvetlen szomszédakkal hasonlítja össze, hanem kezdetben nagyobb lépésközökkel dolgozik. Az algoritmus működése úgy indul, hogy meghatároz egy viszonylag nagy gap értéket (lépésközt), majd a listában olyan elemeket hasonlít össze és rendez, amelyek között ez a gap távolság van. Ez a módszer lehetővé teszi, hogy a rossz helyen levő elemek már a rendezés korai szakaszában messzire kerüljenek a kiindulási pozíciójukból, így a lista gyorsabban közelít a rendezett állapot felé. A gap értéke minden ciklus után csökken – általában a felére –, és amikor végül eléri az 1-et, az algoritmus egy hagyományos beszűrő rendezést hajt végre, de ekkorra a lista már majdnem rendezett, ezért a befejező lépés nagyon gyors. Ennek köszönhetően a ShellSort jelentősen hatékonyabb, mint a BubbleSort vagy a sima insertion sort, és gyakorlati környezetben nagyon jól teljesít, bár pontos futásideje a gap-sorozattól függően változhat.

Mindegyik algoritmus az ékezhelyes `normalize_hu()` függvénnyel dolgozik.

pytofiapp.py

Ez a modul a teljes alkalmazás központja. Összehangolja a menüt, a verskezelőt és a szótárkezelőt.

CL_PytofiApp osztály tartalma:

1. Inicializálás

Betölti:

- a menürendszert (CL_Menu),
- a verskezelőt (CL_PoemManager),
- a szótárkezelőt (CL_DictionaryManager).

2. DEF_add_poem()

- A felhasználó kiválasztja a versfájlt.
- A program beolvassa a fájlból:

- Cím:
- Szerző:
- Dátum:
- A törzsszöveget továbbküldi feldolgozásra a dictionarymanager.py modulnak.

3. DEF_search_word()

Háromféle keresési módot támogat:

1. pontos szavas keresés
2. szórészlet keresés
3. előtag szerinti keresés

A találati lista tartalmazza:

- előfordulások számát,
- mely versekben szerepel.

4. DEF_show_words()

- Betöltött vagy mentett szótár listázása táblázatos formában.

5. DEF_view_poems()

- A felhasználó kiválaszt egy verset,
- A program kiírja a teljes tartalmát.

main.py

Ez a program belépési pontja.

Feladata:

- elindítani az egész alkalmazást a következő hívással: CL_PytofiApp().DEF_run()

Ettől kezdve a főmenü megjelenik, és a felhasználó vezérelheti a program minden funkcióját.

Forrás megjelölés

string műveletek

<https://docs.python.org/3.13/library/string.html#string.punctuation>

os modul

<https://docs.python.org/3/library/os.html>

Pyconio

<https://infopy.eet.bme.hu/pyconio/>

Block karakter

<https://www.compart.com/en/unicode/U+2500>

time modul

<https://docs.python.org/3/library/time.html>

Rendezések

<https://www.geeksforgeeks.org/sorting-algorithms/>

list comprehension

<https://infopy.eet.bme.hu/ea11/#1>